

FinTech Transaction Pipeline

Capstone · Validate → Detect Fraud → Report

Vitalii Kulykivskyi · 2026-07-04

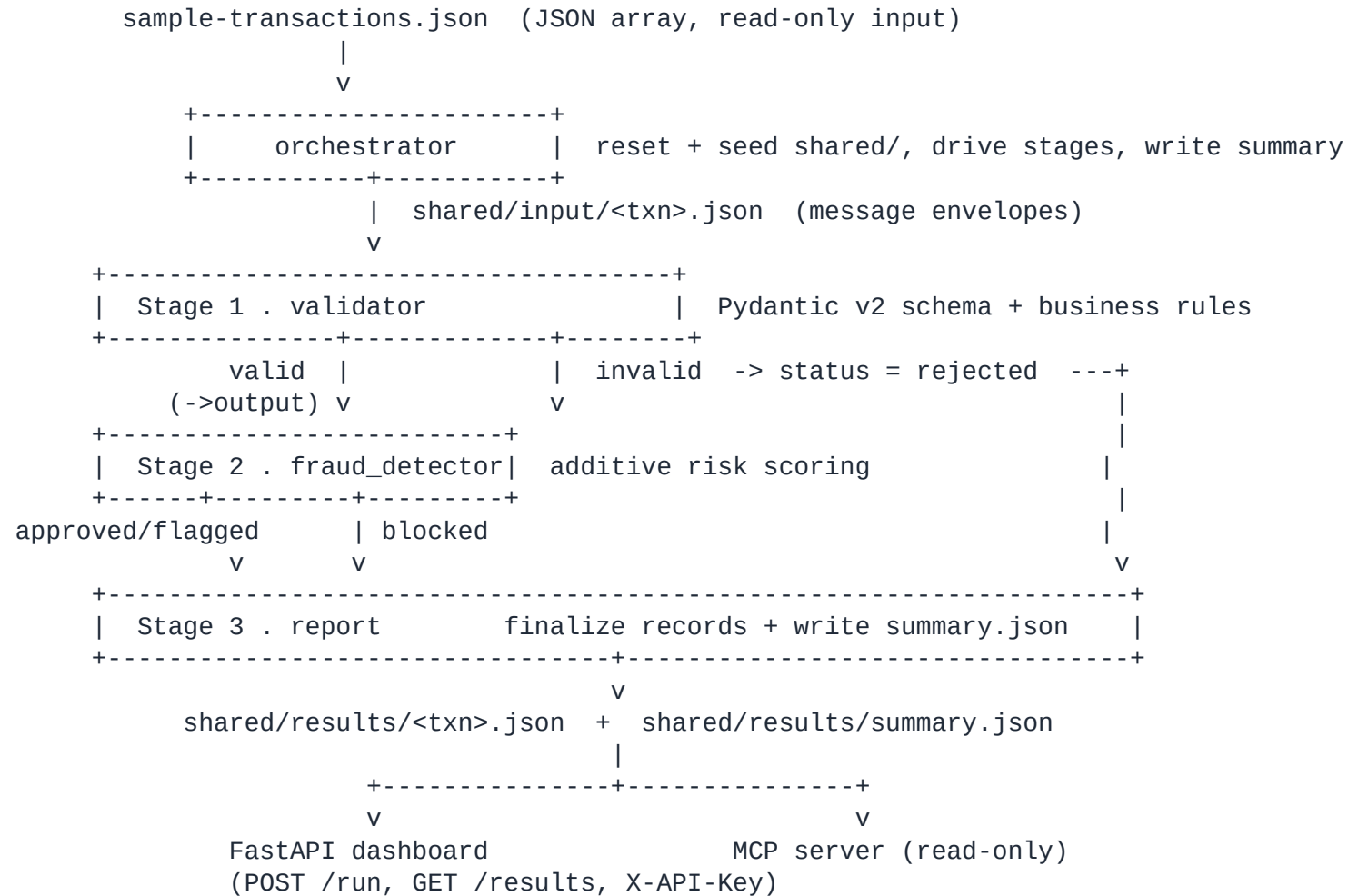
Built with the /generate-pipeline four-agent chain (spec → code → tests → docs)

What it does

A small, file-based pipeline that ingests a JSON batch of financial transactions and takes every record to a single terminal outcome — auditably and idempotently.

- Ingests **sample-transactions.json** (a JSON array of transactions).
- **Validates** each record against a strict schema + business rules.
- **Scores** valid records with an additive fraud rule set.
- **Reports** a per-transaction result plus an aggregate summary.json.
- Every transaction → exactly one status: **approved / flagged / blocked / rejected**.
- Every stage action is written to an append-only, masked audit log.

Architecture — pipeline flow



Every stage --appends--> shared/audit/audit.log (append-only JSONL; accounts masked)

Pipeline stages

- **Orchestrator** — resets & seeds shared/, rejects duplicate ids, drives the 3 stages in order, writes summary.json.
- **Stage 1 · Validator** — Pydantic v2 schema + rules (currency, country, type, positive amount, ISO-8601 ts, distinct accounts).
 - Valid → advance; invalid → terminal **rejected** with reasons.
- **Stage 2 · Fraud detector** — additive risk score over valid records (large / structuring amount, off-hours, cross-border, high-risk destination).
 - Score maps to **approved / flagged / blocked**.
- **Stage 3 · Report** — finalizes terminal records, writes counts to summary.json, exposes the GDPR erasure hook.
- Each stage is a pure `run( . . . )` function with injected directories — framework-free and unit-testable in isolation.

Tech stack

Layer	Technology	Version / note
Language / runtime	Python	3.11+ (tested on 3.14.5)
Validation / domain	Pydantic v2	2.13.4
Dashboard API	FastAPI	0.139.0
ASGI server	Uvicorn	0.50.0
Frontend	Vanilla HTML / CSS / JS	no framework, no build step
Persistence	File-based shared/ tree	JSON records + JSONL audit log
MCP integration	FastMCP	read-only over shared/results/
Testing	pytest / pytest-cov / httpx2	9.1.1 / 7.1.0 / 2.5.0

Demo — batch run

```
$ python orchestrator.py
```

```
Pipeline run complete.
```

```
Total transactions: 8
```

```
approved    3      <- TXN001, TXN004, TXN008
flagged     2      <- TXN002, TXN003   (risk score in review band)
blocked     1      <- TXN005         (fraud score over threshold)
rejected    2      <- TXN006, TXN007   (failed validation)
```

```
# 8 result files + summary.json written to shared/results/
```

```
# every transaction reaches exactly one terminal status
```

Demo — validate without processing

```
$ python -m pipeline.validator --dry-run
```

```
Validation dry-run: sample-transactions.json
```

```
total 8 | valid 6 | invalid 2
```

```
TRANSACTION  RESULT  REASONS
```

```
-----
```

```
TXN001      VALID
```

```
...
```

```
TXN006      INVALID  currency 'XYZ' is not a supported ISO 4217 code
```

```
TXN007      INVALID  amount must be greater than 0 (got -100.00)
```

```
# read-only: never touches shared/ or the audit log
```

Testing & quality gates

- **134 tests, ~96% line coverage** across pipeline, orchestrator, frontend.
- Coverage gate: `--cov-fail-under=80` — enforced in a pre-commit hook.
- Every critical path covered: validation rules, fraud scoring & decision mapping, end-to-end orchestrator run, and API auth (401 / 200).
- Deterministic & isolated — every test uses a `tmp_path` staging tree, so the suite never touches the real shared/ directory.
- New `--dry-run` validator CLI shipped with 10 regression tests (TDD), keeping the suite green.

Compliance by design

Mapped to the FinTech Platform Constitution (CLAUDE.md):

- **Auditability** — append-only JSONL audit log; who / what / when / trace id.
- **Data protection** — account numbers masked in logs; description never logged; GDPR erasure hook (right to be forgotten).
- **Security** — API-key auth on every endpoint; no secrets in code; input validated.
- **Honestly-flagged gaps** — 401 denials not yet audit-logged; erasure hook not HTTP-exposed; local server is plain-HTTP (TLS deferred to prod). Tracked in docs/compliance.md.

How it was built

Produced end-to-end by an orchestrated four-agent chain — each stage independently verified before the next began:

- **1. Specification agent** → specification.md (pinned rules, contracts).
- **2. Code-generation agent** → orchestrator, pipeline/, frontend/ (context7-researched).
- **3. Testing agent** → tests/ at $\geq 80\%$ coverage, all green.
- **4. Documentation agent** → README + docs/ grounded in the real code.
- Orchestrator gates: real pipeline run, real pytest run, real link checks — no stage advanced on an agent's word alone.

Lessons learned

- **Verify, don't trust** — the biggest wins came from gates that re-ran the pipeline and tests independently, catching what a summary would have hidden.
- **Clean boundaries pay off** — framework-free `run( . . . )` stages made both testing (`tmp_path`) and the dry-run CLI trivial to add later.
- **Ground docs in code** — diagrams/tech-stack were derived from the actual wiring & requirements, not memory, so they stayed true.
- **Environment friction is real** — no python on PATH, Windows Store stubs, no PDF binaries: pin an isolated venv and prefer pure-Python tooling.
- **Honesty over polish** — flagging real compliance gaps is more valuable than claiming a perfect score.

Summary & where to look

- 8/8 transactions processed to a terminal status; 134 tests green at ~96% coverage.
- Run it: `HOWTORUN.md` · slash commands: `/run-pipeline`, `/validate-transactions`.
- Architecture & data model: `docs/architecture.md`, `docs/data-model.md`.
- API & compliance: `docs/api.md`, `docs/compliance.md`.

Thank you — questions welcome.